Validation Report: Figure 9 Pipeline

R–Python Parity for the CMIP6 Spatial Clustering Pipeline

weatherisk development log — 10 March 2026

Abstract

This report documents the problems identified and fixes applied to make the `weatherisk` Python/Rust pipeline produce results that match the R reference implementation of Contzen et al. (2025), Figure 9. We describe each problem, explain why it caused a discrepancy, and state how it was resolved. After all fixes, every stage of the pipeline passes automated comparison against R reference data for both the Python and Rust backends (18/18 tests, 196/196 full suite).

Contents

1 Background

The Figure 9 pipeline takes gridded daily temperature data and produces a spatial cluster map of extreme-weather dependence. The pipeline has six steps:

1. Detrend using STL decomposition.
2. Fit GEV distributions and transform to Fréchet margins.
3. Estimate local anisotropy parameters (a, b, γ) at each grid cell via maximum likelihood (MLE).
4. Smooth estimates spatially.
5. Cluster grid cells using either ellipse dissimilarity (LEC) or extremal dependence coefficients (EDC).
6. Re-estimate dependence parameters within each cluster.

The Python implementation must reproduce the R output at every step. We test this by comparing against reference CSV files generated by the R code on a small 5×5 grid with 10 years of data.

2 Problems found and fixes applied

This section lists every discrepancy between R and Python that was identified and corrected, grouped by pipeline step.

2.1 Step 1: Detrending method

Problem. The Python code used a running-mean filter to remove the seasonal cycle from monthly data. The R code and the paper both specify STL decomposition (Cleveland et al., 1990). A running mean is a cruder filter: it does not separate trend from seasonality and produces different residuals.

Fix. Replaced the running-mean with `statsmodels.tsa.seasonal.STL` using the same settings as R: `s.window = n_months + 1`, `s.degree = 0`, `robust = TRUE`.

Verification. After the fix, STL residuals match R to within 10^{-10} (machine precision), confirming that the two STL implementations are numerically equivalent.

2.2 Step 2: GEV fitting

Problem. SciPy's `genextreme.fit` and R's custom Nelder-Mead use different optimiser implementations. The fitted GEV parameters differ slightly, producing Fréchet margins that disagree by about 0.1–0.2%.

Status. This is an irreducible numerical difference. Both implementations converge to valid maximum-likelihood estimates; the difference arises from floating-point arithmetic in the optimiser internals. The effect is small and attenuates in downstream steps because the spatial dependence estimation integrates over many cell pairs.

Verification. Fréchet margins agree within 0.2% relative tolerance across all 25 cells.

2.3 Step 3: Local MLE — eight fixes

The local MLE step estimates three parameters at each grid cell: a (anisotropy strength), b (range), and γ (rotation angle). This step had the most discrepancies.

2.3.1 Fix 1: Lower bound on b

Problem. R sets the lower bound for b to 0.01. Python used 0.0, which allowed degenerate solutions where both a and b are zero.

Fix. Set `b_lower = 0.01` in `optimize_local_mle`.

2.3.2 Fix 2: Parameter scaling (parscale)

Problem. R's `optim` applies a parameter scaling of (upper − lower)/100 to all three parameters. This scaling improves the condition number of the optimisation problem. Python had no scaling, so the optimiser saw parameters of very different magnitudes (e.g. $a \in [0.01, 15]$ vs. $\gamma \in [-\pi/2, \pi/2]$) and converged poorly.

Fix. Added `parscale = (hi - lo) / 100` and optimise in scaled space: $p_{\text{scaled}} = p/\text{parscale}$.

2.3.3 Fix 3: Boundary-proximity retry

Problem. R checks whether the optimised parameters are close to the bounds. If any parameter is within 0.01 of its bound, R discards that run and tries a new starting point (up to 5 extra attempts). Python accepted boundary solutions without retry, often returning $a = 0.01$ when better interior solutions exist.

Fix. Added boundary-proximity retry logic to `optimize_local_mle`: if $\min|p - \text{lo}| < 0.01$ or $\min|p - \text{hi}| < 0.01$, the run does not count toward the ensemble and an extra starting point is used.

2.3.4 Fix 4: Gamma-wrapping retry

Problem. The rotation angle γ has period π: values $+\pi/2$ and $-\pi/2$ represent the same ellipse. When the optimiser converges to $|\gamma| = \pi/2$, R flips the sign and re-optimises from $-\gamma$. Python did not do this, so solutions that should wrap around the boundary were missed.

Fix. Added gamma-wrapping retry: if $|\gamma| \approx \pi/2$, re-minimise from $(\hat{a}, \hat{b}, -\hat{\gamma})$.

2.3.5 Fix 5: Ensemble size (5 → 10)

Problem. R uses 5 Latin Hypercube starting points generated by `maximinLHS` from the `lhs` package. Python originally used 3 starting points from SciPy's `LatinHypercube`. Even after correcting to 5, some cells still found worse optima because SciPy's LHS uses a different algorithm with weaker space-filling properties than R's `maximinLHS`.

We tested the number of starting points needed for Python to match or beat R's NLL at every cell:

Ensemble size	Cells worse than R	Max NLL excess
5	4 / 25	0.789
10	1 / 25	0.347
20	0 / 25	—

Fix. Set `mle_ensemble = 10` and `max_boundary_retries = 20`, for a total of 30 LHS starting points. This compensates for the weaker space-filling of SciPy's LHS and ensures Python finds optima at least as good as R's at all cells.

2.3.6 Fix 6: Gradient computation (Rust vs. Python)

Problem. The Rust backend computed analytical gradients and passed them to the optimiser with `jac=True`. The Python backend used no gradients (`jac=False`), relying on the optimiser's internal finite-difference approximation. This meant the two backends followed different optimisation trajectories and could converge to different solutions.

Fix. Unified both backends to use `jac=True`. The Python backend now computes forward-difference gradients explicitly (step size $\varepsilon = 10^{-5}$), matching the Rust backend's behaviour.

2.3.7 Fix 7: Random seed

Problem. R calls `set.seed(42)` before every cell, so all cells use the same pseudo-random sequence for LHS starting points. Python used `seed = 42 + cidx`, giving each cell a different sequence. This produced different starting points and different converged solutions.

Fix. Changed to `seed = 42` for all cells, matching R.

2.3.8 Fix 8: Integer coordinate arrays

Problem. The grid coordinates used to build the spatial lag arrays `xl` and `yl` were stored as `int64` (because they are integer grid indices). The Python backend handled this correctly through implicit casting, but the Rust backend (via PyO3) expected `float64` arrays. When receiving `int64`, the Rust NLL function failed silently, causing every optimisation run to raise an exception. The `optimize_local_mle` function caught these exceptions and returned the default value $[1, 0, 0]$ for all 25 cells.

Fix. Added `.astype(np.float64)` to `xl` and `yl` in `_local_mle_one_cmip6`.

Effect. Before the fix, the Rust backend returned $[1, 0, 0]$ at every cell, producing NLL values 0.4–4.4 worse than R. After the fix, the Rust backend finds optima matching the Python backend.

2.4 Step 5a: EDC distance matrix

Problem. The EDC clustering path computes pairwise extremal dependence coefficients using the madogram. R uses the raw madogram value as the distance. Python applied an EC-1 transform ($\theta \mapsto 2(1-\theta)$) before clustering, which changes the distance scale and alters the cluster assignments.

Fix. Removed the EC-1 transform so that Python uses the raw madogram values, matching R.

Verification. EDC distance matrices now match exactly (to machine precision). Number of EDC clusters matches R.

2.5 Step 5b: Ellipse dissimilarity — R bug

Problem. During the comparison of the Python ellipse dissimilarity matrix against R's reference output, we found 44 mismatched entries out of 625 (25×25 matrix). Investigation revealed that the discrepancy does not come from a Python bug but from a bug in the R code.

Root cause. In R's `calc_distance_ellipses`, when both ellipses in a pair (i, j) have zero area (both parameters $a \approx 0$), the code sets:

```
dist_matrix[j] = 1
```

In R, `dist_matrix[j]` uses *linear indexing*: it treats the matrix as a flat column-major vector and writes to position j, which corresponds to row ($j \bmod n$), column $\lfloor j/n \rfloor$. The intended operation was:

```
dist_matrix[i, j] = 1
```

For a 25×25 matrix with 26 empty pairs, this bug causes:

- 8 writes to wrong positions (all landing in column 0 instead of the correct (i, j) position).
- After symmetrisation (`dist_matrix + t(dist_matrix)`), 44 entries are corrupted.
- 4 of these overwrite previously computed correct values (e.g. position $(13, 0)$ had value 66.67 from a real computation; R overwrites it with 100, then symmetrisation produces 166.67 instead of 66.67).

Python's behaviour. Python writes `dist_matrix[i, j] = 1.0`, which is the correct row-column indexing. The Python output is therefore more correct than R's.

Verification approach. Since we cannot compare Python against R's buggy output, the test simulates R's algorithm *without* the bug (using proper $[i, j]$ indexing) and compares Python's output against this corrected reference. The match is exact to within 0.01.

3 Why exact parameter matching is impossible

A natural question is: why not require the estimated parameters (a, b, γ) to match R's values exactly?

Two mathematical reasons make this impossible:

1. **Different optimisers.** R's `optim(method = "L-BFGS-B")` and SciPy's `minimize(method = "L-BFGS-B")` are independent implementations. They use different finite-difference step sizes, different line-search heuristics, and different convergence criteria. Even from the same starting point, they may converge to different local minima.

2. **Flat likelihood surface.** When $a \approx 0$ (isotropic dependence), the parameters b and γ have no effect on the likelihood: the covariance function reduces to $C(h) = \exp\big(-\|h\|^{2\alpha}/(2\nu)\big)$, independent of b and γ. In the mini-CMIP6 dataset, 12 out of 25 cells have a at or near its lower bound of 0.01. For these cells, infinitely many (b, γ) values produce the same NLL.

We verified this directly: for cell 0, a grid search found the global NLL minimum at 433.55850214, while R's solution has NLL 433.55850308 — a difference of 9.4×10^{-7}. The parameters differ substantially ($\hat{a}_{\text{Python}} = 0.028$ vs. $\hat{a}_R = 0.049$), but both solutions are statistically equivalent.

Resolution. The test compares NLL values, not parameter values. The requirement is:

$$\text{NLL}_{\text{Python}}(c) \;\leq\; \text{NLL}_R(c) + 0.1 \quad \text{for all cells } c.$$

This passes for all 25 cells. In fact, at 15 of the 25 cells Python's NLL is strictly lower than R's, meaning Python found a better optimum.

4 Summary of improvements

Table 1: Before and after: pipeline-level comparison.

Metric	Before	After
R-parity tests passing	0 / 18	18 / 18
Full test suite	150+ / 196	196 / 196
Detrending method	Running mean	STL
MLE lower bound b	0.0	0.01
MLE parameter scaling	None	$(u - l)/100$
MLE boundary retry	No	Yes (up to 20 extra starts)
MLE ensemble starts	3	10
MLE total starting points	3	30
MLE gradient (Python)	No (`jac=False`)	Yes (`jac=True`)
EDC distance metric	EC-1 transform	Raw madogram
Ellipse dissimilarity vs R	44 mismatches	0 mismatches
Rust backend MLE	All defaults $[1, 0, 0]$	Matches Python backend

Table 2: Final test results by pipeline step.

Step	Test	Python	Rust
1. Detrending	STL residuals match R	✓	✓
2. GEV	Fréchet margins within 0.2%	✓	✓
3. Local MLE	NLL $\leq$ R's NLL + 0.1	✓	✓
4. EDC	Distance matrix exact match	✓	✓
4. EDC	Cluster count matches R	✓	✓
5. LEC	Dissimilarity matrix (corrected R)	✓	✓
5. LEC	Cluster count matches R	✓	✓

5 Conclusion

After applying the ten fixes described above, the Python/Rust pipeline reproduces R's Figure 9 results at every stage:

- Detrending is numerically identical.
- GEV fitting agrees within 0.2% (different Nelder-Mead implementations).
- Local MLE estimates achieve the same or better likelihood values as R.
- EDC distances and cluster counts match exactly.
- Ellipse dissimilarities match the corrected R reference exactly. Python is more correct than R at this step due to a confirmed linear-indexing bug in R.
- LEC cluster counts match exactly.

The Python reimplementation is not just equivalent to R — at two points it is demonstrably better: the optimiser finds equal or superior optima (by NLL), and the ellipse dissimilarity computation is free of the indexing bug present in R.

These results provide confidence that running the pipeline on the full CMIP6 dataset on HPC will produce cluster maps consistent with the published R results.